FRESHFLOW · ENGINEERING CHALLENGE

Order recommendations, served clean

A containerised Python service that loads four messy CSV files and answers one question: what should this store order today?

Eric Värnild · 3 November 2026 · github.com/evarnild/freshflow-engineering-challenge

THE BRIEF

Two endpoints, one container

Load data

Accept the four CSV files (items, orderable items, inventory, recommendations) and ingest them.

Retrieve

Return the order recommendations for a given store and day.

Constraints

Python, any web framework. Runs with plain docker build and docker run.

The real task was hidden in the data: load it naively and the service serves wrong orders.

PROFILING FIRST

The data was dirty on purpose

515

negative recommended quantities (−5 at worst)

332

rows for items 9901–9903: never orderable, not in the catalog, 42 on Sundays

803

inventory dates as dd/mm/yyyy instead of ISO

8

spellings of two store ids: STORE_A, " store_a", "store_b "...

420

exact duplicate rows: 120 inventory, 300 recommendations

1,939

oranges recommended once; the item’s median is 18

THE RULES

Normalise, drop with a reason, or flag

Problem	Action	Why
Store id case, spaces	Normalise	Same store, different spelling
dd/mm/yyyy dates	Normalise	464 have day > 12, none have month > 12
Exact duplicates	Drop, counted	Runs after normalising, or copies survive
Not orderable that day	Drop, counted	Removes 9901–9903 and nothing else
Negative quantity	Serve 0, flag	“Order nothing” is the actionable reading
Outliers, odd inventory	Flag only	A buyer knows if 1,939 is a promotion

Nothing is silently fixed or silently dropped. Every rule has a stable code, a count in the load report, and a test.

ARCHITECTURE

Pure logic in the middle, I/O at the edge

`ingest.py`
normalise → validate → dedupe → cross-file checks

→

`flags.py`
clamping and anomaly flags, named thresholds

→

`query.py`
the store-day view, sorted, joined with inventory

→

`api.py`
FastAPI, uploads, autoload, error mapping

Pure and fully tested

No I/O, no globals. Builds an immutable Dataset and a LoadReport.

I/O

One atomic swap per load

THE API

Five endpoints, explicit errors

Endpoint	Returns
POST /data	the load report
GET /stores/{id}/recommendations?day=	one store’s order list
GET /stores	stores and date ranges
GET /data/report	the last load report
GET /health	whether data is loaded

When something is wrong

- 422: bad file, missing column, invalid CSV, a file that cleans to nothing. The old data stays.
- 413: a file over MAX_UPLOAD_BYTES
- 404: unknown store. 200 and an empty list: a known store with nothing that day.
- 503: nothing loaded yet
- 422: day must be YYYY-MM-DD

THE LOAD REPORT

Every decision is visible to the caller

```
"flags": {
"inventory_suspect": 1523,
"missing_catalog_entry": 606,
"negative_quantity_clamped": 509,
"quantity_outlier": 7
}
```

Per file: rows read, rows accepted, counts per rule code, and up to five example rows for each drop.

The same codes ride on each served row as flags, so a buyer sees why a number looks odd.

RUNNING IT

Two commands, answers straight away

```
docker build -t freshflow .
docker run --rm -p 8000:8000 freshflow
curl "localhost:8000/stores/store_a/recommendations?day=2024-03-05"
```

1.33 s

to load and clean all four files (~25k rows each)

0.27 ms

per store-day query, indexed by (store, day)

284 MB

image, non-root user, data baked in for autoload

HOW IT WAS BUILT

An AI-native SDLC with human gates



I own the gates

Intent, spec and plan accepted before any code. A live check I ran myself before each merge. An agent may never record that one.

Agents do the work

An implementer agent builds the approved plan in its own git worktree, tests first. A separate reviewer agent attacks it.

Tools enforce it

check:intent fails a branch whose code came before its gates. One command, npm run verify, is the definition of done.

REVIEW

Green tests were not enough

Round	Verdict	What it found
1	Rework	A malformed CSV gave 500 and stopped startup; ?day= accepted Unix timestamps; 33 of 87 deliberate bugs went unnoticed by the tests
2	Escalate	All fixed and proven. New: an upload whose rows all drop still wiped the catalog
3	Escalate	My decision: refuse a file with zero accepted rows. 6 of 7 new mutants killed, 1 equivalent

Mutation testing breaks each changed line on purpose and runs the suite. A test that still passes proves nothing.

WHERE IT LANDED

One command says it’s done

122

product tests, including one on the real data

256

tests for the process tooling itself

3

review rounds, each with a live check by me before the merge

```
npm run verify = kit tests + ruff check + ruff format --check + pytest
```

WHAT'S NEXT

The open items are already intents

§	Type	Intent
3	fix	Serve recommended quantities as whole pieces, not 17.0
4	adjustment	Record the 413 upload cap as accepted behaviour
5	refactor	Prove the CSV parser's six remaining edge rules with tests
6	adjustment	Pin the Docker base images by digest
7	adjustment	Add an npm lint script so the SDLC scan sees ruff
8	feature	Dependency audit and secret scan inside npm run verify

Each one comes from a review finding or the SDLC grid, waiting for my intent gate.

Clean in, explained out

A service that never changes a number it can’t explain, built by agents under gates I own.

`github.com/evarnild/freshflow-engineering-challenge`

Questions?